IFT436 - Série d'exercices #6 : diviser-pour-régner

Exercice 1: On a vu que l'algorithme de Karatsuba multiplie deux entiers sur n chiffres en séparant le problème en 3 multiplications d'entiers avec $n/2$ chiffres.

a. Avec plus de travail, on peut séparer le problème en 5 multiplications d'entiers avec $n/3$ chiffres chacun (vous n'avez pas à le faire). Que devient la complexité de l'algorithme sous-jacent?

b. Plus généralement, pour tout entier $k \geq 2$, il est possible de séparer le problème en $2k-1$ multiplications d'entiers avec n/k chiffres chacun. Ensuite, on doit faire $O(k)$ additions, chacune en temps linéaire. Montrez que pour toute constante $\alpha > 1$, on peut multiplier en temps $O(n^\alpha)$.

Solution.

a) Dans ce cas, la récurrence est $f(n) = 5f(n/3) + O(n)$ et on a $a = 5$, $b = 3$ et $c = 1$. Puisque $\log_3(5) > 1$, on est dans le cas 1 du théorème maître, qui nous dit que la complexité est $O(n^{\log_3 5})$.

b) Pour toute constante $k \geq 2$, la récurrence correspondante est $f(n) = (2k-1)f(n/k) + O(n)$. Notez que l'algorithme sous-jacent devra faire $O(k)$ additions et que le $O(n)$ utilise le fait que k est une constante. Il est clair que $\log_k(2k-1) > 1 = c$ et donc la complexité de l'algorithme sera $O(n^{\log_k(2k-1)})$.
Par la puissance de Wolfram Alpha, on a que $\lim_{k\to\infty} \log_k(2k-1) = 1$ (exercice: prouvez-le sans Wolfram). Ceci veut dire que pour tout $\alpha > 1$, il existe k tel que $\log_k(2k-1) < \alpha$. Donc, étant donné α, on choisit une constante k telle que $\log_k(2k-1) < \alpha$, on applique l'algorithme de la question et on atteint une complexité $O(n^\alpha)$.

Exercice 2: Soient A et B deux matrices de dimension $n \times n$, où n est une puissance de 2. On veut calculer $A \times B$, c'est-à-dire la multiplication de A et B. L'approche classique prend un temps $O(n^3)$.
L'algorithme de Strassen fait mieux. Nous n'allons pas le détailler ici, mais vous pouvez consulter `https://fr.wikipedia.org/wiki/Algorithme_de_Strassen`.
L'algorithme arrive à séparer le problème en 7 multiplications de matrices de taille $n/2 \times n/2$, en plus de faire $O(1)$ additions et soustractions de matrices, chacune en temps $O(n^2)$.

a. Quelle est la complexité sous-jacente à l'algorithme de Strassen?

Solution.

La récurrence est $f(n) = 7f(n/2) + O(n^2)$. On a $a = 7, b = 2, c = 2$ et $\log_2 7 > 2 = c$. On est dans le cas 1 du théorème maître et la complexité est $O(n^{\log_2 7})$, ce qui est environ $O(n^{2.81})$.

b. Le nombre d'additions et soustractions de matrices faites à chaque récursion est au plus 18. Lorsque $n = 1$, aucune addition/soustraction n'est faite.

Montrez que si n est une puissance de 2, le nombre total d'additions/soustractions faites par l'algorithme est au plus $3 \cdot (7^{\log n} - 1)$

Solution.

On ne peut pas utiliser le théorème maître ici, car on cherche une quantité exacte au lieu d'une quantité en terme de O. Soit $f(n)$ le nombre d'additions faites sur une matrice de taille $n \times n$. On a

$$f(n) \leq \begin{cases} 0 & \text{si } n = 1 \\ 7f(n/2) + 18 & \text{si } n > 1 \end{cases}$$

On prouve par induction que $f(n) = 3 \cdot (7^{\log n} - 1)$. Pour le cas de base, avec $n = 1$ on a $f(n) = 0$ et $3 \cdot (7^{\log n} - 1) = 3 \cdot (7^0 - 1) = 3 \cdot (1 - 1) = 0$. Pour l'induction, on suppose que pour tout $m < n$ puissance de 2, on a $f(m) \leq 3 \cdot (7^{\log m} - 1)$. Pour n, on a

$$\begin{aligned} f(n) &\leq 7f(n/2) + 18 \\ &\leq 7 \cdot (3 \cdot (7^{\log(n/2)} - 1)) + 18 \quad \text{(par hypothèse d'induction)} \\ &= 21 \cdot 7^{\log(n/2)} - 21 + 18 \\ &= 21 \cdot 7^{\log n - \log 2} - 3 \\ &= 21 \cdot 7^{\log n - 1} - 3 \\ &= 21 \cdot 7^{\log n}/7 - 3 \\ &= 3 \cdot 7^{\log n} - 3 \\ &= 3 \cdot (7^{\log n} - 1) \end{aligned}$$

Exercice 3: On nous donne un tableau d'entiers T avec un intervalle $[a \,..\, b] = \{a, a+1, \ldots, b]$ et on veut savoir combien il y a de paire dans T dont la somme est dans $[a \,..\, b]$.
Entrée: tableau d'entiers $T = [t_1, t_2, \ldots, t_n]$ et deux entiers a, b.
Sortie: nombre de paires $\{i, j\}$ avec $i \neq j$ telles que $a \leq t_i + t_j \leq b$.

a. Donnez le pseudo-code d'un algorithme diviser-pour-régner qui implémente la stratégie suivante: on sépare T en deux sous-tableaux de taille $n/2$; on calcule le nombre de paires recherchées récursivement sur les deux moitiés; on trie les deux moitiés de tableau; on calcule le nombre de paires recherchées qui ont un nombre dans la moitié gauche et un nombre dans la moitié droite.

Solution.

Il y a en fait plusieurs façons de faire. Ici, je présente une approche qui divise en moitiés X et Y et trie Y. Puis, pour chaque $x \in X$, on utilise la recherche dichotomique pour trouver le premier élément $Y[i]$ de Y tel que $x + Y[i] \geq a$, et le dernier élément $Y[j]$ tel que $x + Y[j] \leq b$ (si vous ne l'avez jamais vu, faites en exercice le travail d'implémenter cette recherche dichotomique). Le nombre de paires concernant x est $j - i + 1$.

On suppose qu'on a accès à une routine $rechdicho_borne(T, min, max)$ qui retourne les positions (i, j) telles que $T[i \ .. \ j]$ est le sous-tableau de T contenant les valeurs entre min et max. Si T est trié, ceci peut s'implémenter en temps $O(\log n)$.

fonction *getNbPaires(*T, a, b*)*
  si $|T| = 1$ **alors**
    return 0
  $X = T[1 \ .. \ n/2]$
  $Y = T[n/2 + 1 \ .. \ n]$
  $nbX = getNbPaires(X, a, b)$
  $nbY = getNbPaires(Y, a, b)$

  $nbXY = 0$
  trier Y avec $triFusion$
  pour $x \in X$ **faire**
    $(i, j) = rechdicho_borne(T, a - x, b - x)$
    $nbXY = nbXY + (j - i + 1)$
  fin
  return $nbX + nbY + nbXY$

b. Donnez la complexité de l'algorithme de la question précédente.

Solution.

L'exécution d'un appel de *getNbPaires* s'exécute en temps $O(n \log n)$. En effet, le tri est en temps $O(n \log n)$ et la boucle exécute une recherche dichotomique $n/2$ fois, chaque itération en temps $O(\log n)$. Donc cette boucle prend un temps $O(n \log n)$, ce qui justifie le temps discuté.
La récurrence est

$$f(n) = 2f(n/2) + O(n \log n)$$

Pour les paramètres du théorème maître, on a $a = b = 2$, $c = 1$ et $k = 1$. On est dans le cas 2, dans quel cas la complexité devient $O(n^c \log^{k+1} n) = O(n \log^2 n)$.

c. (BONI) Donnez un algorithme diviser-pour-régner en temps $O(n \log n)$ pour ce problème.

Solution.

Pas facile! Il faut qu'un appel s'exécute en temps $O(n)$ au lieu de $O(n \log n)$. Je donne les idées à haut niveau. On peut demander aux appels récursifs de nous retourner X et Y déjà triés. Pour que notre appel retourne une version triée de T, il suffit d'ajouter un boucle de fusion sur X et Y et de le retourner à l'appel parent. Ceci prend un temps $O(n)$.

Ensuite, si on a X et Y triés, il faut trouver le nombre de paires voulues entre X et Y. On prend $X[1]$ et on trouve (i, j) tel que $Y[i..j]$ a tous les éléments dont la somme avec $X[1]$ est dans $[a..b]$ (temps $O(\log n)$, que l'on exécute une seule fois). Ensuite, on passe à $X[2]$. Si $X[2] + Y[j] > b$, on décrémente j jusqu'à ce qu'on trouve $X[2] + Y[j] \leq b$. On décrémente aussi i pour trouver le plus petit i tel que $X[2] + Y[i] \geq a$. On continue avec $X[3]$, et ainsi de suite. Ceci fonctionne parce que X et Y sont triés.

Les valeurs de i et j ne font que décrémenter, ce qui veut dire qu'on les décrémentera au plus n fois chacun. Le temps total passé dans la boucle décrite ci-haut prend donc un temps total $O(n)$. On aura la même récurrence que le tri fusion, ce qui mène à un algorithme $O(n \log n)$.

Si vous voulez plus de détails, venez en discuter.

Exercice 4: Considérez l'algorithme artificiel suivant:

fonction *algoArtificiel(T) //T est un tableau de taille n*
$A = algoArtificiel(T[1\ ..\ n/5])$
$B = algoArtificiel(T[2n/5\ ..\ 3n/5])$
$traitementQuelconque(A, B)$
$C = algoArtificiel(T[n/5\ ..\ 2n/5])$
$D = algoArtificiel(T[4n/5\ ..\ n])$
$traitementQuelconque(C, D)$
$E = algoArtificiel(T[3n/5\ ..\ 4n/5])$
$F = algoArtificiel(T[1.5n/5\ ..\ 2.5n/5])$
$G = algoArtificiel(T[2.5n/5\ ..\ 3.5n/5])$
return $fonctionQuelconque(A, B, C, D, E, F, G)$

En supposant que la fonction *traitementQuelconque* s'effectue en temps $O(n)$ et $fonctionQuelconque$ en temps $O(1)$, donnez la complexité de cet algorithme.
Et si *traitementQuelconque* s'effectuait en temps $O(n^{1.3})$?

Solution.

Il suffit d'utiliser le théorème maître. Le problème se divise en $a = 7$ sous-appels sur des instances de taille $n/5$, et donc $b = 5$. Si le traitement est $O(1) = O(n^0)$, on a $c = 0$. Puisque $\log_b a > c$, la complexité est $\Theta(n^{\log_5 7})$.
Si le traitement est $O(n^{1.3})$, il s'avère que $\log_5 7 \leq 1.21$, et ici selon le théorème maître la complexité est $O(n^{1.3})$.

Exercice 5: Considérez le problème du sous-tableau de somme maximum:

Entrée: un tableau $T = [t_1, \ldots, t_n]$ d'entiers positifs ou négatifs.
Sortie: un sous-tableau contigu $T' = [t_i, t_{i+1}, \ldots, t_{i+k}]$ tel que la somme $\sum_{j=i}^{i+k} T[j]$ est maximum parmi tous les choix possibles.

Donnez un algorithme en temps $O(n)$ pour ce problème.
Indice: il y a plusieurs façons d'y arriver. En particulier, il y a une méthode diviser-pour-régner. Pour combiner les sous-solutions, les sous-appels, en plus du sous-tableau maximum, pourraient retourner d'autres informations.

Solution.

Notre algorithme va retourner plus de valeurs qu'il n'en faut. Il nous faut quelques définitions.

a. On appelle la *m somme maximum* la somme des élément d'un sous-tableau contigu de valeur maximum (donc, ce qu'on veut obtenir à la fin). La somme peut être 0 si le tableau est vide.

b. On appelle la *p somme maximum préfixe* la somme maximum possible d'un sous-tableau contigu de T tel que $T[1]$ est dans ce sous-tableau (donc la meilleure somme possible en commençant du début).

c. On appelle la *s somme maximum suffixe* la somme maximum possible d'un sous-tableau contigu de T tel que $T[n]$ est dans ce sous-tableau (donc la meilleure somme possible en commençant par la fin).

d. On appelle la *t somme totale* la somme de tous les éléments de T.

Mis à part la somme totale, toutes ces valeurs pourraient être à 0 si on décide de n'inclure aucun élément. Nous n'allons pas nous attarder à trouver les indices des sous-tableaux - seuls les valeurs vont nous intéresser, mais l'algorithme pourrait être modifié facilement.
L'algorithme retourne (m, p, s, t), où m est la somme maximum, p la somme maximum préfixe, s la somme maximum suffixe et t la somme totale. Je recommande de vous faire un dessin pour bien comprendre comment chacune des valeurs est calculée.

fonction *sousTabMax(T)*
  si $|T| = 1$ **alors**
    si $T[1] < 0$ **alors**
      return $(0, T[1], T[1], T[1])$
    sinon
      return $(T[1], T[1], T[1], T[1])$
  $(m_1, p_1, s_1, t_1) = sousTabMax([t_1, \ldots, t_{n/2}])$
  $(m_2, p_2, s_2, t_2) = sousTabMax([t_{n/2+1}, \ldots, t_n])$
  //On calcule maintenant (m, p, s, t) à retourner
  $t = t_1 + t_2$
  $p = \max(p_1, t_1 + p_2)$
  $s = \max(s_2, t_2 + s_1)$
  $m = \max(m_1, m_2, s_1 + p_2)$
  return (m, p, s, t)

La récurrence pour cet algorithme donne $f(n) = 2f(n/2) + O(1)$. Selon le théorème maître, ceci est $O(n)$.

Exercice 6: Dans le problème de la **majorité**, on reçoit un tableau T d'objets quelconques et on veut savoir si un élément est strictement majoritaire. On suppose que $|T| = n$ est une puissance de 2, et on veut savoir s'il existe un élément x qui est présente au moins $n/2 + 1$ fois dans T.

a. Si vous pouvez trier les éléments de T, i.e. $x < y$ est bien défini sur les objets de T, donnez un algorithme en temps $O(n \log n)$ pour le problème de la majorité.

Solution.

On trie, puis on parcourt de gauche à droite en comptant le nombre d'occurrences de l'élément courant. On garde en mémoire l'élément qui apparaît le maximum de fois, et on retourne true s'il est présent $n/2 + 1$ fois ou plus.

b. Supposons maintenant que $x < y$ n'est pas défini, i.e. vous avez des objets incomparables. Par contre, vous pouvez vérifier l'égalité, donc pour deux éléments de T, vous avez accès à $x = y$ qui retourne $true$ si x et y sont égaux, et $false$ sinon.

Donnez un algorithme en temps $O(n \log n)$ pour le problème de la majorité. Considérez une méthode diviser-pour-régner.

Solution.

L'observation principale est que si m est un élément majoritaire, alors il doit être majoritaire dans le tableau gauche ou bien le tableau droit (ou les deux).

fonction *majo*$(T = [t_1, \ldots, t_n])$
  si $n = 1$ **alors**
    return $T[1]$
  $m_1 = majo([t_1, \ldots, t_{n/2}])$
  $m_2 = majo([t_{n/2+1}, \ldots, t_n])$
  si $m_1 \neq nil$ **alors**
    $c_1 =$ nombre d'occurrences de m_1 dans T $(O(n))$
    si $c_1 > n/2$ **alors**
      return m_1
  si $m_2 \neq nil$ **alors**
    $c_2 =$ nombre d'occurrences de m_2 dans T $(O(n))$
    si $c_2 > n/2$ **alors**
      return m_2
  return nil

La récurrence pour cet algorithme est $f(n) = 2f(n/2) + O(n)$, ce qui est $O(n \log n)$.

c. (BONI, PLUS DIFFICILE) Donnez un algorithme en temps $O(n)$ qui vérifie si T contient un élément majoritaire. La méthode diviser-pour-régner ne fonctionne peut-être pas ici.

Solution.

La réponse facile: avec une table de hachage H, vous pouvez traverser le tableau de gauche à droite et faire $H[T[i]]+ = 1$ pour chaque élément $T[i]$. Vous comptez ainsi le nombre d'occurrence de chaque élément en temps $O(n)$ avec forte probabilité, et il vous suffit de retourner true si un élément apparaît $n/2+1$ fois ou plus. On peut faire mieux, mais c'est long à expliquer! J'invite les intéressés à regarder l'algorithme de majorité de Boyer-Moore: `https://en.wikipedia.org/wiki/Boyer%E2%80%93Moore_majority_vote_algorithm`.
Expliqué en termes simples ici: `http://codinghelmet.com/exercises/majority-element`.

Exercice 7: Vous avez un algorithme récursif dont l'analyse en temps donne la récurrence suivante:

$$f(n) = \begin{cases} 1 & \text{si } n = 0, 1 \\ 2f(n-1) + 8f(n-2) + O(1) & \text{sinon} \end{cases}$$

Donnez la valeur de f en terme de complexité O.

Solution.

Soit $f(n)$ le nombre de sommets dans l'arbre de récursion, au total. La racine représente l'appel $f(n)$, et il y a 10 enfants: deux représentent un appel à $f(n-1)$, et 8 enfants un appel à $f(n-2)$.

On applique la recette vue en classe. On pose $f(n) = 2f(n-1)+8f(n-2)$. On devine que $f(n) = r^n$ et on a

$$r^n = 2r^{n-1} + 8r^{n-2}$$

Ceci mène à

$$r^2 = 2r + 8$$

puis à

$$r^2 - 2r - 8 = 0$$

On peut factoriser en

$$(r+2)(r-4) = 0$$

et les racines sont donc 4 et -2. Donc toutes les solutions ont la forme $f(n) = c_1 4^n + c_2(-2)^n$.
Ceci qui est $O(4^n)$. Si on prenait le temps de trouver les constantes, on verrait que c'est aussi $\Theta(4^n)$.
Donc, le nombre de noeuds de l'arbre de récusion est $O(4^n)$, et chaque appel prend un temps $O(1)$, et ainsi la complexité est $O(4^n)$.

Exercice 8: Soit $G = (V, E)$ un graphe. Un ensemble $C \subseteq V$ est appelé une *clique* si $\forall u, v \in C, uv \in E$. Donc toutes les arêtes possibles entre les éléments de C sont présentes.
Dans le problème de la clique maximum, on cherche la clique $C \subseteq V$ de taille maximum.
Considérez la stratégie suivante: on construit une clique C un sommet à la fois, avec C initialement vide. On prend un sommet $u \in V$, et on branche récursivement sur deux possibilités: soit $u \in C$, ou bien $u \notin C$. On retire ensuite u de G, et on continue récursivement sur les deux cas. Les deux versions de C sont passées récursivement, et quand on atteint un cas terminal,

on a une clique qu'on retourne.
Écrivez le pseudo-code qui implémente cette stratégie. Donnez ensuite une récurrence qui exprime le temps requis par cet algorithme, puis évaluez sa complexité.

Solution. Nous présentons deux solutions. La première correspond à l'intuition donnée dans l'énoncé. La deuxième utilise la notion d'ensemble indépendant.
Notre premier algorithme va recevoir la clique courante C, initialement vide. Supposons que $V(G) = \{v_1, \ldots, v_n\}$,et donc qu'un ordre des sommets est connu. On reçoit aussi un indice i nous disant que nous en sommes à considérer le i-ème sommet. À l'appel initial, $C = \emptyset$ et $i = 1$. L'idée est de considérer les sommets v_i un par un et de brancher sur deux possibilités: soit v_i est ajouté à la clique courante (si possible), soit v_i n'est pas ajouté.

fonction *maxClique*$(G = (V, E), C, i)$
  si $i > |V|$ **alors**
    return C
  $v_i =$ le i-ème sommet de G
  $C_1 = C$
  si v_i *est voisin de tous les sommets de* C **alors**
    $C_1 = maxClique(G, C \cup \{v_i\}, i + 1)$
  $C_2 = maxClique(G, C, i + 1)$
  si $|C_1| > |C_2|$ **alors**
    return C_1
  sinon
    return C_2

Le temps de cet algorithme peut être estimé comme suit. Une exécution de l'algorithme prend un temps $O(n)$ pour vérifier si v_i est un voisin de tous les sommets de C. La complexité est donc proportionnelle à n, fois le nombre de branchements effectués. Ce nombre de branchements est donné par

$$f(n) = 2f(n-1)$$

car on fait deux appels récursifs avec un sommet de moins à considérer. En devinant $f(n) = r^n$, on obtient $r^n = 2r^{n-1}$ et donc $r = 2$. Donc, $f(n) = c2^n \in O(2^n)$.
Au final, la complexité est donc $O(n \cdot f(n)) = O(n2^n)$.

Solution. Pour la deuxième solution, on définit un ensemble indépendant comme un sous-ensemble $X \subseteq V(G)$ tel que pour tout $u, v \in X$ distincts, $uv \notin E(G)$.
Soit $\overline{G}$ le complément de G, c'est-à-dire qu'on inverse les arêtes et non-arêtes de G. En d'autres termes, $V(\overline{G}) = V(G)$ et $E(\overline{G}) = \{uv : u, v \in V(G)\} \setminus E(G)$. Il est facile de voir qu'une clique de taille maximum correspond à un ensemble indépendant de taille maximum dans $\overline{G}$. On va donc s'intéresser au problème de trouver un ensemble indépendant de taille maximum dans un graphe.
Considérez l'algorithme suivant. Ici, $G - v$ dénote le graphe duquel on retir v, et $G - N(v) - v$ le graphe duquel on retire v et tous ses voisins $N(v)$.

fonction *indset*$(G = (V, E))$
  si $|E| = 0$ **alors**
    return V
  Soit v un sommet de G de degré maximum
  $I_1 = indset(G - v)$
  $I_2 = indset(G - N(v) - v) \cup \{v\}$
  si $|I_1| > |I_2|$ **alors**
    return I_1
  sinon
    return I_2

Le principe de l'algorithme est que pour le v choisi, il y a 2 cas. Soit v est dans la solution optimale, ou bien v n'est pas dans la solution optimale. Si v n'y est pas, on n'a qu'à retirer v et trouver une solution optimale sur $G - v$. Si v est dans la solution optimale, alors on ne peut pas ajouter ses voisins (par définition d'un ensemble indépendant). Dans ce cas, il faut chercher une solution sur $G - N(v) - v$.
Pour la complexité, on note qu'il faut un temps $O(n)$ dans chaque appel pour trouver v et calculer $G - v$ et $G - N(v) - v$. L'appel $G - v$ se fait sur un graphe avec $n - 1$ sommets. On note que le v choisi a au moins un voisin, car sinon on aurait $|E| = 0$. Ceci veut dire que $G - N(v) - v$ a $n - 2$ sommets ou moins. La complexité de cet algorithme est donc représenté par la récurrence

$$f(n) \leq f(n-1) + f(n-2) + O(n)$$

on a déjà vu que le nombre de noeuds de l'arbre de récursion de cette fonction est $O(1.618^n)$. Le temps total de l'algorithme est borné par $O(1.618^n \cdot n)$.

Exercice 9: Considérez l'algorithme suivant.

fonction *algoBidon*($S = \{s_1, \ldots, s_n\}$) //*S est un ensemble de taille n*
  si $n \leq 10$ **alors**
    return S
  pour $i = 1..8$ **faire**
    $S' = S$ après avoir retiré les éléments s_i, s_{i+1}, s_{i+2}
    $T_i = algoBidon(S')$
  fin
  return $fonctionMagique(T_1, T_2, T_3, T_4, T_5, T_6, T_7, T_8)$

Supposez que S' peut être construit en temps $O(1)$, et que $fonctionMagique$ prend un temps $O(1)$.
En terme de O, combien de noeuds l'arbre de récursion de cet algorithme contient-il?

Solution.

On voit que chaque traitement dans cet algorithme récursif prend un temps $O(1)$. Il faut déterminer le nombre de noeuds de l'arbre de récursion.
On voit que l'algorithme fat 8 appels récursifs, chacun avec trois éléments en moins passés à la récursion. Donc, le nombre de récursions est proportionnel à

$$f(n) = 8f(n-3)$$

On pose $f(n) = r^n$. On obtient

$$r^n = 8r^{n-3}$$

et donc

$$r^3 = 8$$

et donc $r = 2$ (il y a deux autres racines dans les nombres complexes, mais on les ignore).
On déduit que cet algorithme fait $O(2^n)$ récursions. Chaque appel est $O(1)$, et donc la complexité est $O(2^n)$.